

An Application of Causality and symbolic computation from Boolean networks to a compiler

Alberto Hernández-Espinosa

September 2026

Abstract

The Luminal challenge asks for a compiler backend that schedules a straight-line program on a small VLIW machine and places its values in a shared scratchpad, so that the program runs in as few cycles and as few scratch words as possible. We answer it with two compilers. The first is a strong *classical* solution, critical-path list scheduling followed by first-fit allocation, which we treat as our benchmark. The second is the compiler we propose, which takes a causal view of the problem: it asks which few decisions are responsible for the current cost, reopens exactly those decisions, and solves them again exactly, keeping a checkable reason for every option it rules out. On the eight public programs the classical compiler scores 1.901379 and ours 2.187957, a gain of 15.1%, against the serial baseline’s score of one. Ours is better on 7 programs and equal on 1, and it is correct on every supplied case. The gain comes from memory: it reduces scratch by 3.316 times against the classical 2.420 times, while giving up a little in cycles (1.444 against 1.494 times). The price is compile time: our compiler spends its budget of 0.1 s, where the classical one needs well under a millisecond; both are far inside the grader’s allowance of 20 s.

The answer in four lines. (1) *Baseline*: the serial compiler, score one by definition. (2) *Classical benchmark*: critical-path list scheduling and aligned first-fit allocation, score 1.901379. (3) *Our proposal*: build a correct first answer from exact queries, then repeatedly reopen the decisions that cause the cost and re-solve them exactly, score 2.187957. (4) *Trade-off*: 15.1% higher score and 24.5% lower geometric-mean $C \times S$ than classical, paid for with roughly 0.1 s of compilation per program.

1 The problem in one page

Luminal builds compilers for AI inference, where performance depends on how tightly work is packed in time and how little memory it occupies while it waits [2]. The take-home challenge [1] is a small, exact version of that problem. A program is a list of operations in dependency order: loads, scalar and vector arithmetic, selects and stores. The machine issues one *bundle* per cycle, and each bundle may contain up to two load, two scalar and two vector operations, one store and one flow operation. An operation’s result can be read only after its *latency*: one cycle for an add, three for a scalar load, four for a vector load. Every result lives in a scratchpad of 256 words of 32 bits; vectors take eight consecutive words starting at a multiple of eight. Two values may share words only if their lifetimes do not overlap.

The compiler must decide two things for every operation: *when* it issues (its cycle) and *where* its result lives (its scratch address). Two numbers measure the result: the number of cycles C , and the footprint S , the highest scratch word used. The score compares both with a frozen serial compiler,

$$\text{score} = \sqrt{\underbrace{\text{geomean}_p \frac{C_p^{\text{serial}}}{C_p}}_{\text{cycle speedup}} \times \underbrace{\text{geomean}_p \frac{S_p^{\text{serial}}}{S_p}}_{\text{scratch reduction}}}, \quad (1)$$

so time and memory weigh exactly the same. For a single program, improving the score is the same as reducing the product

$$J = C \times S. \quad (2)$$

This observation shapes our whole approach: a cycle saved and a proportional amount of scratch saved are worth the same, so a good compiler should be willing to trade one for the other whenever the product falls.

The two decisions pull against each other. Packing a schedule tightly makes many values alive at once, which raises S ; packing memory tightly forces values to wait for each other, which raises C . This coupling is what makes the problem interesting, and it is where a greedy compiler leaves the most on the table.

2 Three compilers, three ideas

A program small enough to watch. To show how a compiler actually decides, we follow one public program of the challenge, `02_scalar_dual_chain`, through every compiler in this section and the next. It has 11 operations and 9 values, all scalars:

0	<code>a0 = load a[0]</code>	<i>one word from buffer a (latency 3)</i>
1	<code>b0 = load b[0]</code>	<i>one word from buffer b (latency 3)</i>
2	<code>c0 = load c[0]</code>	<i>one word from buffer c (latency 3)</i>
3	<code>d0 = load d[0]</code>	<i>one word from buffer d (latency 3)</i>
4	<code>left_product = a0 * b0</code>	<i>(latency 2)</i>
5	<code>right_product = c0 * d0</code>	<i>(latency 2)</i>
6	<code>cross = a0 + c0</code>	<i>shared by both chains (latency 1)</i>
7	<code>left = left_product + cross</code>	<i>(latency 1)</i>
8	<code>right = right_product ^ cross</code>	<i>(latency 1)</i>
9	<code>out[0] = left</code>	<i>one store slot per cycle</i>
10	<code>out[1] = right</code>	<i>one store slot per cycle</i>

The line number is the operation’s identifier, the one used by every figure below. Two chains share one value. Nothing in the program says in which order the four loads must happen, how the chains should interleave, or which word each value should occupy; those are exactly the decisions being scored.

Every compiler below is shown in the same picture (Figure 1), which replays the compiled program on the machine one cycle at a time. Each column is a cycle. The top strip names the operations issued in that cycle: that is the *schedule*. Below it there is one row per scratch word, and a cell holds the number stored in that word at the end of the cycle: that is the *allocation* at work. A green cell marks a value landing in its word, which happens a latency after the operation that produced it was issued. The bottom rows are the buffers, the only memory the grader reads; the answer appears there when the stores execute. **The number of columns is C , the number of scratch rows is S .** Every picture was produced by running the compiler and replaying its output, and each replay is checked against the challenge’s own simulator before it is drawn.

The baseline: serial. The challenge supplies a serial compiler that issues one operation per bundle and gives every value a fresh address. It is always correct, never good, and it defines a score of one. On the public programs (programs provided as part of the challenge to test development) the supplied starter compiler produces exactly the same schedules and footprints. In Figure 1A every column holds at most one operation, and every value keeps its own word to the end, long after its last reader has finished with it: $C = 12$, $S = 9$, $J = 108$.

The classical benchmark. Our classical compiler is a version based on optimization techniques found in literature. It schedules with a *critical-path list scheduler* [3, 4]: at each cycle it issues

the ready operations that head the longest remaining chain of latencies, filling every free slot. It then computes each value’s live interval and places values by *aligned first fit* [5]: each value takes the lowest address that is free for its whole lifetime, vectors on multiples of eight. It tries three fixed orders of the values and keeps the best. This compiler is fast, simple and strong; it scores 1.901379. Its limitation is structural: it decides once, in order, and never revisits a decision. It also optimises time first and memory second, even though the score weighs them equally.

Figure 2 shows its decisions on the example. At cycle 0 all four loads are ready and all four have the same priority, 6, the latency of the longest chain from each load to the end of the program. The tie is broken by fan-out: `a0` and `c0` each feed two operations, their product and `cross`, so they take the two load slots, and `b0` and `d0` wait for cycle 1. The rule is reasonable locally, and its consequences are visible in Figure 1B. Neither multiply can start until `b0` and `d0` land at cycle 4, so the two chains advance in lockstep and finish together; their two stores then compete for the single store slot, and one of them waits, at cycles 7 and 8. Memory pays as well: at cycle 4 all four inputs and `cross` are alive at once, so no allocation can use fewer than five words, and first fit uses exactly five. The result is $C = 9$, $S = 5$, $J = 45$. The allocation is optimal for the schedule it was given; what costs a cycle and two words is the schedule itself, chosen by a rule that never sees memory.

Our proposal: reopen and resolve. The compiler we propose starts from a different question. Instead of asking “what is the best next step?”, it asks “which decisions are responsible for the cost I have now, and is there any way to set them differently that lowers J ?” It answers that question exactly, for small groups of decisions at a time, and repeats. We call this *reopen and resolve*; internally the final version is named C1.

The idea comes from causal deconvolution in algorithmic information dynamics [9, 10, 11]: an observed outcome is explained by separating it into the contributions of the mechanisms that produced it, and each mechanism is studied by intervening on it while the rest is held fixed. Here the outcome is a compiled program, its cost is $C \times S$, and the mechanisms are groups of decisions. The last operations to issue produce C ; the values alive at the memory peak produce S . Reopening one such group while freezing everything else is an intervention, and solving it exactly tells us whether that group can do better. The compiler does not run a general deconvolution algorithm; it borrows this causal stance and applies it to scheduling.

One ingredient makes this practical: the set of legal choices for a group of decisions is kept as an exact and compact description, which can be asked questions such as “what is the smallest legal cycle?” or “is any assignment with J below the current value legal?” without listing its members one by one. Section 3 shows how that description is written, how a question is asked of it, and how each answer becomes a decision. On the example it reaches $C = 8$, $S = 3$, $J = 24$, against 45 for classical and 108 for serial, and it proves that nothing better exists.

3 Our approach, step by step

We follow `02_scalar_dual_chain`, one of the public provided programs, and follow it through every step. The compiler works by a single loop: it *writes a question* about a set of options, *obtains the answer* exactly, and *turns the answer into a decision*. Step 1 asks one question per decision; Steps 2 to 5 ask questions about groups of decisions.

Step 1: a correct first answer, one question per decision. Operations are taken in program order, and for each the compiler asks: *which cycles may this operation use, given everything decided so far?* The answer is a set of cycle numbers, and the way the set is written is what makes the question cheap.



Figure 1: How a compiler's output runs, cycle by cycle, on 02_scalar_dual_chain. Columns are cycles. The top strip is the schedule (operations issued); the middle band is the allocation (one row per scratch word, with the value it holds); the bottom band is main memory. Green: a value landing in its word, or a store reaching memory. (A) Serial, 12×9 . (B) Classical, 9×5 : the loads are issued in two pairs, the four inputs and **cross** are all alive at cycle 4, and the last store waits a cycle for the store slot.

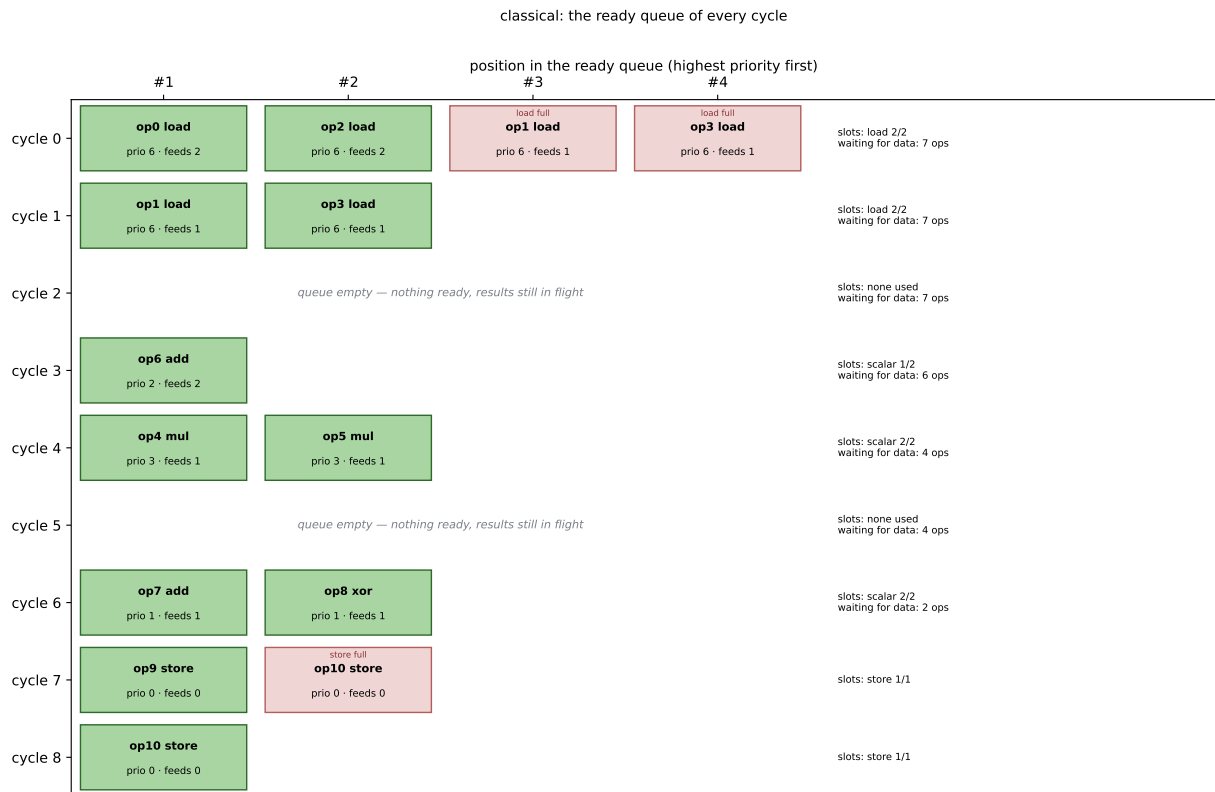


Figure 2: The classical list scheduler, decision by decision. Each row is one cycle's ready queue, sorted by priority (the longest latency chain to the end, then fan-out). Green cards take a slot; red cards find their engine full and wait for the next cycle. The tie among the four loads at cycle 0 is broken in favour of the two loads that feed two operations each.

Our compiler obeys exactly the same rules as the classical one, precedence and engine slots; what differs is where the rules are applied. The classical compiler applies them cycle by cycle: at each cycle it inspects every waiting operation, checks whether its inputs have landed and a slot is free, ranks the ready ones and issues them. Ours applies them once, when it writes the question. Before asking about an operation it computes the earliest cycle at which its inputs have landed (for operation 4, `left_product`, that is cycle 3: its inputs `a0` and `b0` are loaded at cycle 0 with latency 3), and this becomes the lower end of the window. Cycles whose engine is already full are then removed. The set that remains contains legal cycles only, so its smallest member is both legal and as early as possible. Step 1 still decides one operation at a time and never revisits an answer; each answer becomes a constraint on the questions that follow. Treating decisions jointly is the work of Steps 2 to 5.

The *binary format* has three ingredients, shown on the example in Table 1 and Figure 3.

(a) *Cycle numbers as bits, and the horizon.* Every cycle number is written in binary. How many bits are needed depends on the latest cycle a schedule could ever require. Issuing each operation only after the previous one has completely finished is always legal, so no schedule needs more cycles than the sum of all latencies. For the example, that sum is the sum of the all latencies for the all operations, $3 + 3 + 3 + 3 + 2 + 2 + 1 + 1 + 1 + 1 + 1 = 21$, which here we call the *horizon*: no operation ever needs a cycle beyond 20. Five bits name the 32 cycles 0 to 31, so 5 bits suffice. The same reasoning closes each operation’s own window: it ends at the sum of the latencies of the operations before it, which is why the four loads have windows ending at 0, 3, 6 and 9 (Table 2). We write the bits lowest first, as the implementation does: the first character is x_0 , of weight 1, and the last is x_4 , of weight 16. Cycle 6 is therefore 01100 ($2 + 4$).

(b) *Cubes.* A *cube* is a bit pattern in which some positions are fixed, to 0 or 1, and the others, marked *, are free. It stands for every number obtained by filling the free positions in all possible ways, so a pattern with k free positions contains exactly 2^k numbers. The name comes from Boolean algebra and logic minimisation: the 2^5 five-bit strings are the corners of a five-dimensional cube, a pattern with no free position is one corner, and a pattern with k free positions is a k -dimensional face of it, a sub-cube. Thus 00000 is the single cycle 0, and **000 is a square face holding cycles 0, 1, 2 and 3. The implementation stores a cube as two integers, an *anchor* (the values of the fixed bits) and a *mask* of the free positions (Figure 3). Setting every free bit to 0 gives the anchor, so *the smallest member of a cube is its anchor*, read without listing anything.

(c) *Covers.* A single cube can only describe an aligned block whose size is a power of two, such as $\{0, 1, 2, 3\}$ or $\{4, 5\}$. An arbitrary range of cycles is therefore written as a union of disjoint cubes, a *cover*. The 7 cycles from 0 to 6 are $\{0, 1, 2, 3\} \cup \{4, 5\} \cup \{6\}$, that is **000, *0100 and 01100, in the same way that $7 = 4 + 2 + 1$. The number of cubes grows with the number of bits, not with the length of the range: any range of n -bit numbers needs at most about $2n$ cubes, and no question of the example needs more than 4. Removing cycles and intersecting ranges are operations between cubes, and the smallest member of a cover is its smallest anchor.

One question, answered. Take operation 2, the load of `c0` (Table 1 and Figure 4). A load reads no earlier value, so its window starts at cycle 0 and ends at 6: the three cubes **000, *0100 and 01100. Cycle 0 already holds two loads, operations 0 and 1, and the load engine has two slots, so cycle 0 must go. Removing it is a set difference, not a bit flip: **000 minus 00000 leaves 1*000, the odd cycles 1 and 3, and 01000, the cycle 2. The bits are the digits of the cycle numbers, not one flag per cycle; the leading 1 of 1*000 says “odd”, not “used”. Nor is a cycle removed merely because something issues in it. It is removed only for operations on the same engine, and only once that engine’s slots are all taken: cycle 1, holding one load, still accepts a second (operation 3 goes there), and a scalar operation could still use cycle 0. The anchors of the surviving cubes are 1, 2, 4 and 6, so the answer is cycle 1, and the decision follows at once: operation 2 issues at cycle 1. Every member of the surviving cover is legal, so the decision is legal by construction, and the lost cycle has a stated cause.

Cycle	Bits $x_0x_1x_2x_3x_4$	Cube in the window	Cube after the removal
0	00000	**000	<i>removed: engine full</i>
1	10000	**000	1*000
2	01000	**000	01000
3	11000	**000	1*000
4	00100	*0100	*0100
5	10100	*0100	*0100
6	01100	01100	01100

Table 1: The notation on one question (operation 2, load c0). Each cycle of the window is written with its own five binary digits, lowest first. The window is a cover of three cubes. Removing the full cycle 0 splits the first cube into the odd cycles, 1*000, and the one even cycle left, 01000; the other two cubes are untouched. Generated from the run.

one cube of the example, taken apart x0..x4:1*000 anchor=1 free_mask=2 size=2					
coordinate	x0	x1	x2	x3	x4
weight 2^{**i}	1	2	4	8	16
anchor bits	1	0	0	0	0
free mask bits	0	1	0	0	0
printed label	1	*	0	0	0

the sumandos	anchor	x1 (+2)	=	member
	1	+0	=	1
	1	+2	=	3

every subset of the 1 free weights, added to the anchor: 2 of 2 members drawn

Figure 3: Anatomy of one cube, 1*000, from the example. Top: each position x_i has weight 2^i ; the anchor bits give the fixed part and the mask marks the free position. Bottom: every member is the anchor plus a choice of free weights, here $1 + 0$ and $1 + 2$, the cycles 1 and 3. The smallest member is the anchor.

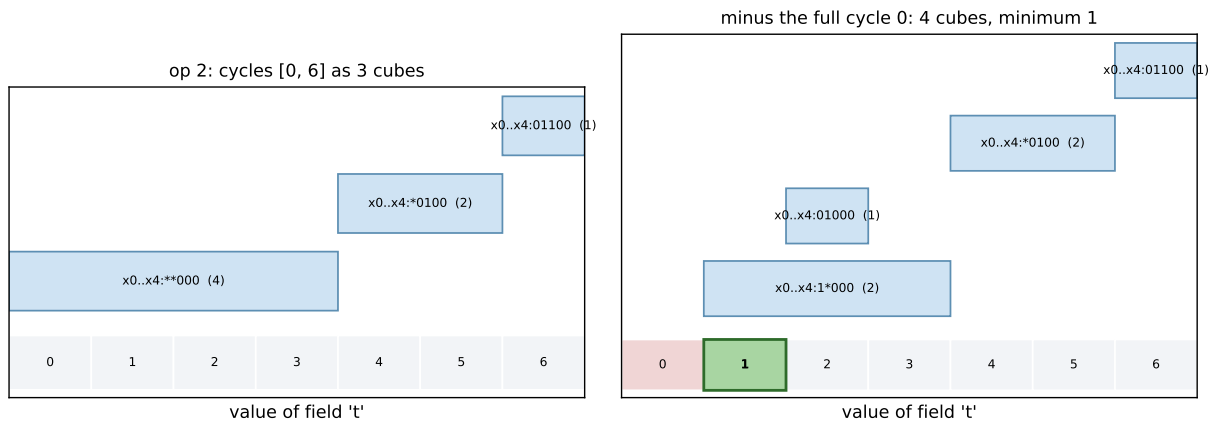


Figure 4: One scheduling question, in the binary format. Operation 2 (load c0). Left: the window of cycles it may use, written as three cubes (bits lowest first; each bar spans the cycles its cube contains). Right: the full cycle 0 (red) removed; the cover now has four cubes and its smallest member, cycle 1 (green), is the decision.

The same question is asked for every operation (Table 2, top, and Figure 5, top). Operations that read earlier values cannot use any cycle before their inputs land; those cycles lie outside the window. The loads of `a0` and `b0` take cycle 0, those of `c0` and `d0` take cycle 1, and every other operation issues in the first cycle at which its inputs have landed and its engine has a free slot.

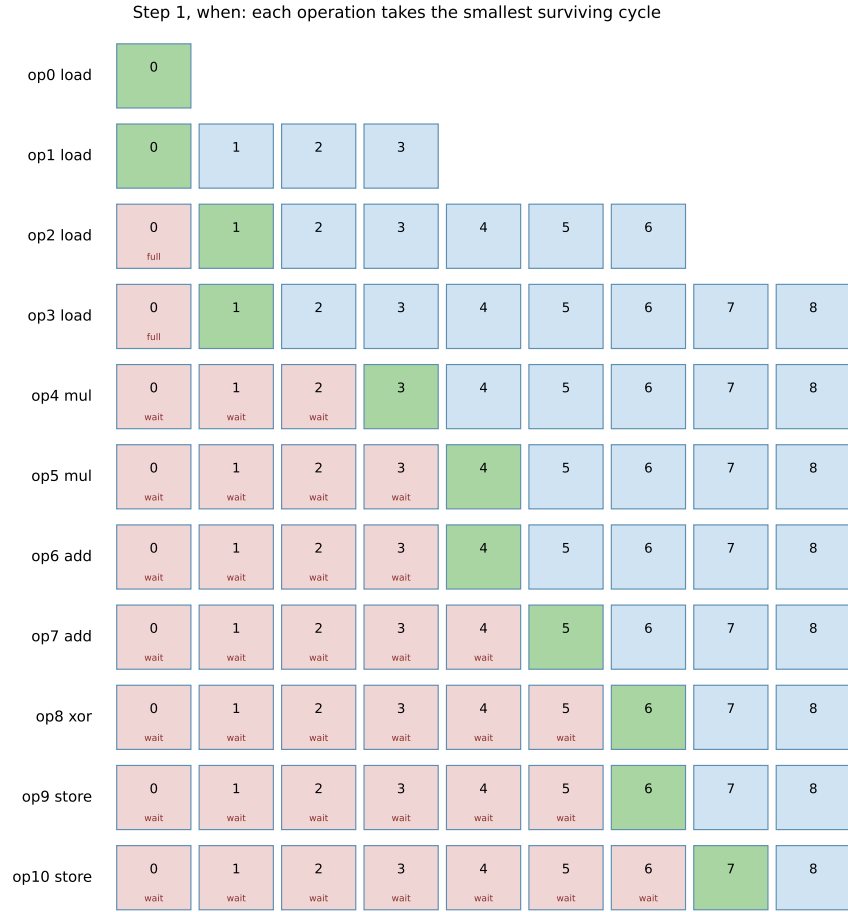
Addresses are chosen by the same loop, on 8 bits (the 256 words of the scratchpad): *which words may this value occupy?* The window is written as cubes, the words held by values whose lifetimes overlap this one are subtracted, and the lowest survivor is the decision (Table 2, bottom, and Figure 5, bottom). For instance `d0` lives only in cycle 4; at that moment `a0` holds word 0 and `c0` holds word 1, so both are removed and `d0` takes word 2. Values whose lifetimes have ended leave their words free, which is how `left_product` reuses word 0 once `a0` has been read for the last time. In all, 20 questions build the whole first answer.

Op	Opcode	Window	Surviving cover (bits lowest first)	Removed	Cycle
0	load	[0, 0]	00000	–	0
1	load	[0, 3]	**000	–	0
2	load	[0, 6]	1*000, 01000, *0100, 01100	0	1
3	load	[0, 9]	1**00, 01*00, 00100, *0010	0	1
4	mul	[3, 12]	11000, **100, **010, 00110	–	3
5	mul	[4, 14]	**100, **010, *0110, 01110	–	4
6	add	[4, 16]	**100, ***10, 00001	–	4
7	add	[5, 17]	10100, *1100, ***10, *0001	–	5
8	xor	[6, 18]	*1100, ***10, *0001, 01001	–	6
9	store	[6, 19]	*1100, ***10, **001	–	6
10	store	[7, 20]	11100, ***10, **001, 00101	–	7

Value	Lifetime	Window	Words held by an overlapping value	Word
<code>a0</code>	[3, 4]	[0, 0]	–	0
<code>b0</code>	[3, 3]	[0, 1]	0 (<code>a0</code>)	1
<code>c0</code>	[4, 4]	[0, 1]	0 (<code>a0</code>)	1
<code>d0</code>	[4, 4]	[0, 2]	0 (<code>a0</code>), 1 (<code>c0</code>)	2
<code>left_product</code>	[5, 5]	[0, 0]	–	0
<code>cross</code>	[5, 6]	[0, 1]	0 (<code>left_product</code>)	1
<code>right_product</code>	[6, 6]	[0, 1]	1 (<code>cross</code>)	0
<code>left</code>	[6, 6]	[0, 2]	0 (<code>right_product</code>), 1 (<code>cross</code>)	2
<code>right</code>	[7, 7]	[0, 0]	–	0

Table 2: Step 1 on the example: every question and its answer. Top: one scheduling question per operation. The window starts where the operation’s inputs have landed; “removed” lists the cycles whose engine was already full; the answer is the smallest member of the surviving cover. Bottom: one allocation question per value. A value’s lifetime runs from the cycle its result lands to the cycle of its last reader; words held by values alive at the same time are removed, and the answer is the lowest word left. Generated from the run.

The result, and what it means. Figure 6 replays the first answer. It runs in $C = 8$ cycles with $S = 3$ words, $J = 24$, against $9 \times 5 = 45$ for classical. Because the loads were taken in pairs by chain, the left chain starts one cycle before the right one: the two chains are staggered rather than in lockstep, their stores fall at cycles 6 and 7 and never compete for the store slot, and at no cycle are more than three values alive. Reading the middle band of Figure 6 row by row shows the allocation at work: word 0 holds `a0`, then `left_product`, then `right_product`, then `right`, each arriving only after the previous tenant’s last read. We should be candid about why Step 1 does so well here: it takes operations in program order and the earliest cycle for each, and on this program that order happens to be a good one. It is not always so; on the program of Section 4, Step 1 alone is worse than classical. That is why the compiler does not stop here.



Step 1, where: each value takes the lowest word no live value holds



Figure 5: Step 1 as pictures. Top: for each operation, every cycle from 0 to the end of its query window. Red cells are unavailable: *wait* marks cycles before its inputs land, which lie outside the window, and *full* marks cycles removed from the window because the engine's slots are taken; blue cells survived; green is the smallest survivor, the decision. Bottom: the same for scratch words, one row per value with its lifetime; a red word carries the name of the live value that holds it.

ours: C = 8 cycles, S = 3 words, J = 24

issued	0,1	2,3	.	4	5,6	7	8,9	10
scratch 0				a0 13	13	left_product 65	right_product 189	right 159
scratch 1				b0 5	c0 21	cross 34	34	34
scratch 2					d0 9	9	left 99	99
a[0]	13	13	13	13	13	13	13	13
b[0]	5	5	5	5	5	5	5	5
c[0]	21	21	21	21	21	21	21	21
d[0]	9	9	9	9	9	9	9	9
out[0]	0	0	0	0	0	0	99	99
out[1]	0	0	0	0	0	0	0	159
	0	1	2	3	4	5	6	7
	cycle							

Figure 6: Our compiler’s output on the example, in the same picture as Figure 1: 8×3 . The chains are staggered by one cycle, the stores never collide, and three words are reused by all 9 values.

Step 2: find the decisions that cause the cost. The compiler builds a *catalogue* of groups of related operations, called windows, each allowed to move its issue cycles within a radius while everything outside the window stays fixed. The catalogue deliberately targets the causes of cost: the k latest operations, which determine C ; the k operations around the memory peak, which determine S ; contiguous blocks; small groups of four; and the whole program when it is small. This is large neighbourhood search [6]: move several related decisions together, because useful improvements rarely come one decision at a time. Each window becomes one question: *if only these decisions may change, is there any legal answer with a smaller J?* On the example the catalogue holds 17 such questions (Figure 7, left), from windows of a single operation up to the whole program.

Step 3: say exactly what “better” must look like. With the rest of the program fixed, a window’s answer cannot use fewer than L_C cycles or fewer than L_S words. If the current cost is J_0 , a strict improvement must satisfy $C \times S \leq J_0 - 1$, and therefore

$$C \leq \left\lfloor \frac{J_0 - 1}{L_S} \right\rfloor, \quad S \leq \left\lfloor \frac{J_0 - 1}{L_C} \right\rfloor. \quad (3)$$

Every improvement lies inside this box, so the search is confined to it without losing any. If a bound falls below its minimum, the window provably holds no improvement and is answered without searching. This is how the objective of Equation (2) enters the search directly: the compiler never asks for fewer cycles or fewer words in isolation, only for a smaller product.

On the example $J_0 = 24$. For the k4 memory window the fixed part of the program already forces $L_C = 8$ and $L_S = 3$, so the box is $C \leq 7$ and $S \leq 2$; both bounds lie below their minimums, the box is empty, and the question is answered by arithmetic alone. 7 of the 17 questions end this way. For the whole-program window the minimums are $L_C = 7$ and $L_S = 1$, and the box is $C \leq 21$ (the horizon) and $S \leq 3$: any improvement must use at most 3 words and fit under the curve $C \times S = 24$ (Figure 7, right).

Step 4: delete impossible options, each with a reason. Before searching, the compiler removes choices that cannot be part of any legal answer. If v reads the result of u and u has latency ℓ , then v cannot issue before u ’s earliest cycle plus ℓ , and u cannot issue after v ’s latest cycle minus ℓ . If an engine’s slots in a cycle are all taken, that cycle is removed from every other operation on that engine. Every removal records a short *certificate* that anyone holding the program can check in one line, in the spirit of explained propagation in constraint solvers [8]. These are the causes in the causal reading: not only “this option is gone”, but “this option is gone because v reads u with latency ℓ ”. On the example, 8 further questions are closed here, before a single search node: at the root, propagation together with the lower bound on J already rules out every combination inside the box, which proves the window holds no improvement.

Step 5: search exactly, keep the first validated improvement, repeat. What remains is searched exactly, depth first, abandoning any branch whose best possible J is no better than the current one. A controller keeps 8 windows open at once and gives each a short turn of at most 2 ms, so no single hard window can consume the budget. The first window to find a strict improvement wins: its answer is checked by the challenge’s own validator, becomes the new current answer, and a new round starts from it. The submitted compiler stops after 0.1 s or when a whole round finds nothing. This is the combinatorial view of scheduling and allocation of Castañeda Lozano and colleagues [7], applied to small, well-chosen neighbourhoods rather than to the whole program at once.

On the example only 2 questions survive to the search: the window of the 16 latest operations and the whole-program window, which on a program of 11 operations release the same operations with different radii. Both are searched to the end, in 366 nodes in total (186 for the whole-program window), and neither contains a strict improvement. The round finishes having found nothing, so the compiler stops after 1 round, long before its budget, and returns the first answer unchanged, together with 15,137 certificates that justify every option it discarded. This agrees with an independent check that shares nothing with the search: enumerating all 24,142 legal schedules of at most 11 cycles (the only ones that could reach $J \leq 23$, because every two-operand operation needs two words at once, so $S \geq 2$) finds none below 24. On this program our answer is optimal, and the classical one is 1.875 times the optimum.

Why this came out better. The example isolates the three differences. First, the classical scheduler decides by a local, time-only rule: the tie among the loads was broken by fan-out, and that one choice put both chains in lockstep, which cost a cycle at the store slot and two words at the memory peak. Steps 2 to 5 judge every question by J itself, so a schedule that saves memory is never discarded for failing to save time. Second, each of our decisions is the exact answer to a question about a set of legal options, written compactly as cubes, so no legal option inside a question is overlooked and nothing illegal is ever produced; every removal has a stated cause. Third, our compiler knows when to stop looking: here, a single round of 366 nodes proves that no window, including the whole program, can do better, which is a guarantee the classical compiler cannot give. On this program Step 1 already found the optimum and Steps 2 to 5 certified it; on the program of Section 4 it is Steps 2 to 5 that turn a first answer worse than classical into one that beats it.

An engineering refinement. A search node differs from its parent by a single decision, yet a naive implementation re-checks every constraint at every node. The final version keeps, at each node, the facts its parent had already derived and re-examines only the constraints that the new decision can reach. Measured on 200 fresh programs at identical search work (10,000 nodes), it makes exactly the same decisions as the version without the refinement in 1,000 of 1,000 paired runs, and its compile call takes 0.574 of the time (97.5% interval [0.552, 0.598]). Because it is

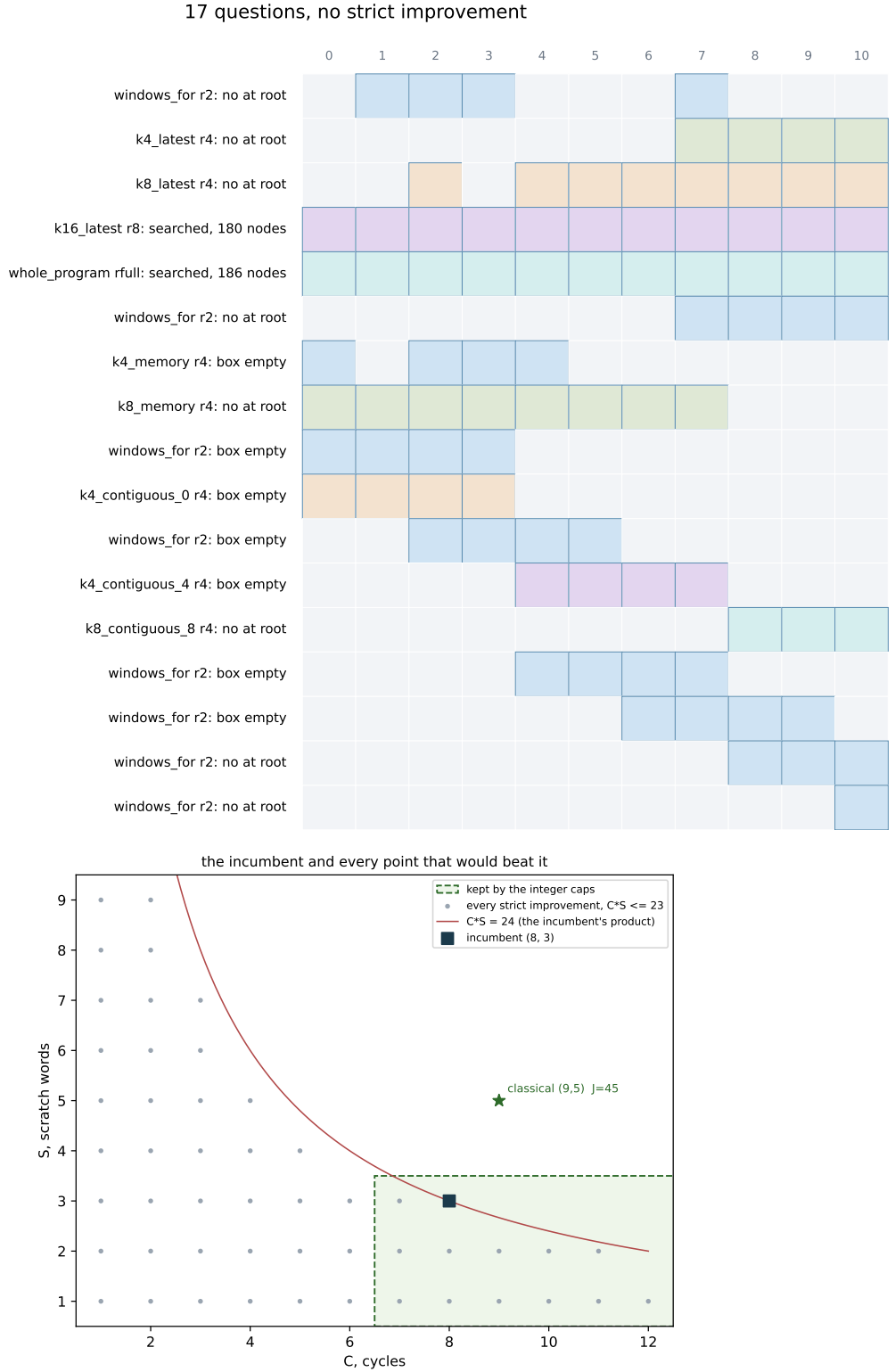


Figure 7: Reopen and resolve on the example. Left: the 17 questions of the round, one row per window; a filled cell is an operation that question may move, and the label gives its verdict (box empty, closed at the root by propagation, or searched to the end). Right: the (C, S) plane. Grey dots are all points with $C \times S \leq 23$, that is every strict improvement; the curve is $C \times S = 24$; the dashed box is the whole-program question's region. The search proves that no legal compilation lies in it.

faster, it explores more within the same wall-clock budget; at 0.1 s its geometric-mean J is 0.991 of the unrefined version’s (interval $[0.985, 0.996]$).

4 A worked example: 05_mixed_broadcast

We now follow one public program from the challenge repository through every compiler. Figure 8 shows it: 8 operations, three loads, two **splats** that copy a scalar into all eight lanes of a vector, a vector multiply, a vector add and a vector store. Its difficulty is easy to see. The vector load is slow, several eight-word vectors are alive together, and the order in which the vectors are created decides how much scratch they need.

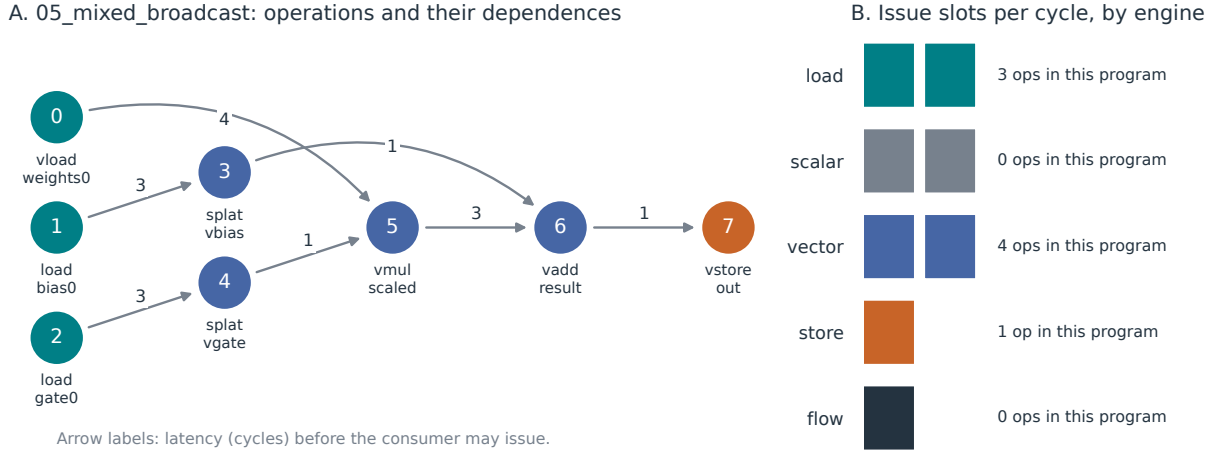


Figure 8: The worked example. (A) Operations and their dependences; each arrow is labelled with the latency that must pass before the consumer may issue. (B) Issue slots per cycle for each engine.

Serial. One operation per cycle and a fresh address for everything: $C = 11$, $S = 42$, $J = 462$. This is the yardstick.

Classical. The list scheduler issues the three loads as early as the two load slots allow, starts the vector work as soon as inputs arrive, and first fit reuses words as soon as values die: $C = 9$, $S = 17$, $J = 153$. It is a big improvement over serial, and on this program the greedy choices are good.

Our Step 1. The first answer takes, for every operation, the earliest legal cycle; for instance operation 2 could issue anywhere in cycles 0 to 7, cycle 0 is already full on the load engine, and so it takes cycle 1. It then gives each value the lowest legal address. The result is $C = 10$, $S = 24$, $J = 240$: legal, but *worse than classical*. Issuing every vector as early as possible keeps two eight-word vectors alive together, and that costs memory. A compiler that stops here loses to the classical one on this program.

Our Steps 2 to 5. The compiler now reopens. It builds a catalogue of 11 windows (Figure 9B). For the window that releases the latest operations, the fixed part implies at least 9 cycles and 8 words, and the box of Equation (3), with the cycle bound also limited by the search horizon, becomes $C \leq 17$ and $S \leq 26$. Inside it, the declared combinations of issue cycles number 7,873,200; propagation with 5 certificates reduces them to 972,000 before any search begins. The search then walks the path shown in Figure 9C:

- the first round, through the window of the latest operations, keeps the same number of cycles and cuts the footprint: 10×17 ;
- the second round, through the same window, saves a cycle: 9×17 , level with classical;
- the third round, through a small window of four operations that had nothing to offer at the start, saves one more word: 9×16 .

The final answer, $C = 9$, $S = 16$, $J = 144$, beats the classical compiler. It took 4 rounds and 5,476 search nodes, and every intermediate answer was validated by the challenge’s checker. Notice the pattern: a large window opened the way and a small one finished it. Neither kind of move alone reaches this answer.

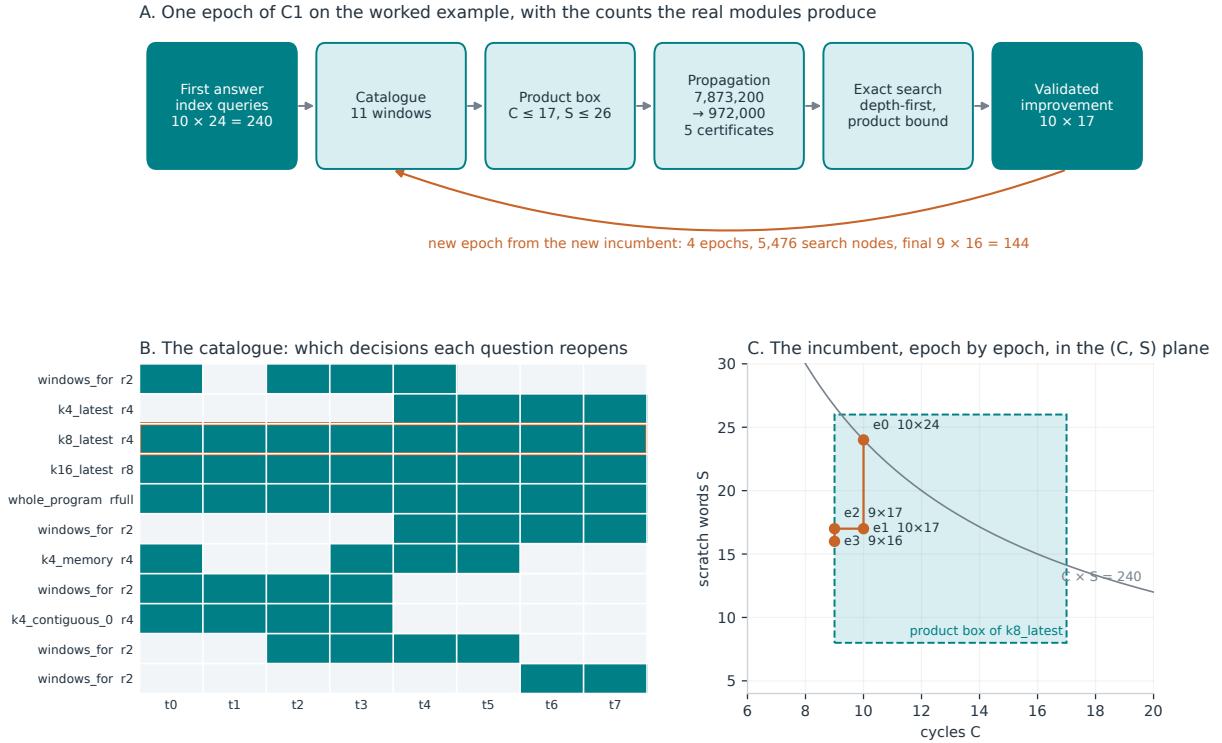


Figure 9: Reopen and resolve on the worked example. (A) One round, with the counts that the real compiler produces. (B) The catalogue: each row is one window and each filled cell an operation it may move. (C) The current answer, round by round, in the (C, S) plane; the curve is the starting value of J , the shaded region the box of Equation (3).

Figure 10 puts the schedules and memory layouts side by side. The difference between classical and ours is visible in the lower panels. Both keep the vectors inside two eight-word blocks; the difference lies in the scalars. The classical layout places one scalar above the vector blocks, which raises the footprint by one word. Ours issues the scalar loads so that each scalar lives in words that no vector occupies at that moment, and the footprint closes exactly at the top of the second block.

5 Results against the classical benchmark

5.1 The public programs

Table 3 gives every public program. Our compiler is better than classical in J on 7 programs and equal on 1; it is never worse. Its geometric-mean J is 24.5% below classical, and its score is 2.187957 against 1.901379.

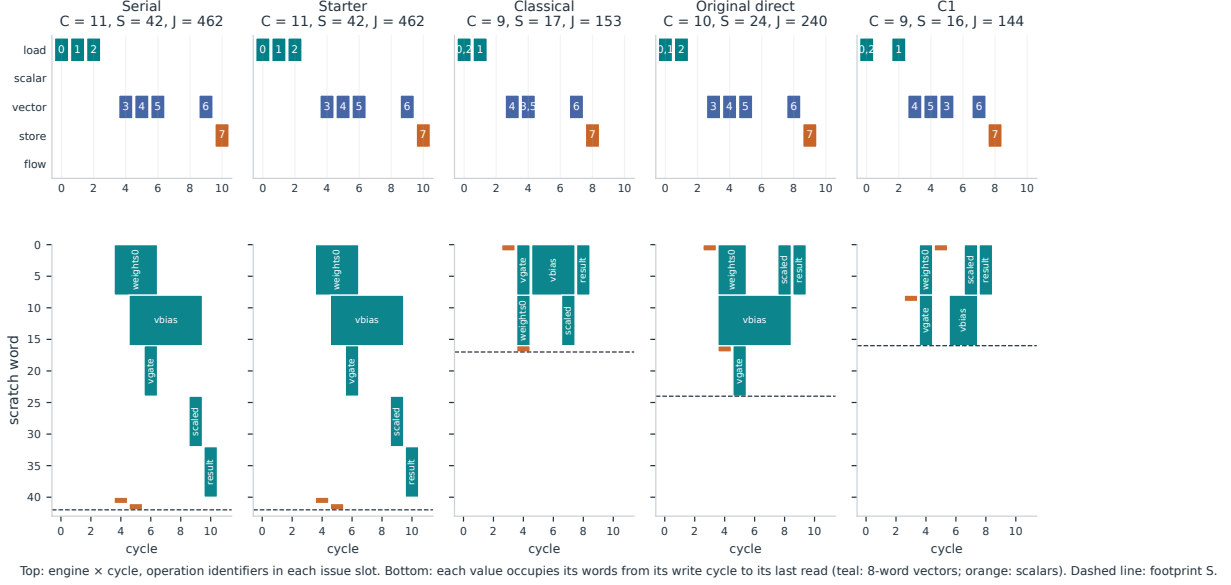


Figure 10: Every compiler on the same program. Top: engine by cycle, with the operations issued in each slot. Bottom: scratch words by cycle; each value occupies its words from its write to its last read, and the dashed line is the footprint S . “Original direct” is our Step 1 alone; “C1” is the full compiler. Every layout was produced by running the compiler and checked by the challenge’s validator.

Program	$C \times S$			J		
	Serial	Classical	Ours	Serial	Classical	Ours
scalar_pipeline	18 $\times$ 15	14 $\times$ 6	14 $\times$ 6	270	84	84
scalar_dual_chain	12 $\times$ 9	9 $\times$ 5	8 $\times$ 3	108	45	24
vector_axpy	14 $\times$ 73	10 $\times$ 32	11 $\times$ 16	1,022	320	176
vector_bitmix	16 $\times$ 89	10 $\times$ 40	13 $\times$ 24	1,424	400	312
mixed_broadcast	11 $\times$ 42	9 $\times$ 17	9 $\times$ 16	462	153	144
parallel_memory	21 $\times$ 128	14 $\times$ 56	13 $\times$ 40	2,688	784	520
scalar_selects	20 $\times$ 16	9 $\times$ 6	10 $\times$ 5	320	54	50
vector_reduction	19 $\times$ 137	12 $\times$ 40	12 $\times$ 32	2,603	480	384
Cycle speedup over serial				1	1.494	1.444
Scratch reduction over serial				1	2.420	3.316
Combined score (Equation 1)				1	1.901379	2.187957

Table 3: The eight public programs. “Ours” is the submitted configuration, with a 0.1 s search budget; bold marks the lowest J in each row. Every value was identical in each of 5 fresh-process repetitions, and every compilation passed the challenge’s validator on all supplied cases.

The split between the two halves of the score is the most instructive part of the table. Our compiler uses fewer scratch words than classical on 7 of the eight programs, but fewer cycles on only 2, and more cycles on 3. On `vector_axpy`, for example, it accepts one more cycle to halve the footprint. That is not an accident; it is the objective at work. Because the score weighs time and memory equally, one extra cycle on a program of about ten cycles is a good trade for half the memory, and a compiler that searches the product $C \times S$ finds such trades, where one that optimises cycles first cannot see them.

5.2 Programs never seen during development

The public programs were visible while we built the compiler, so they show exact results on eight fixed programs, not an estimate of performance on unseen ones. To test generalisation we generated fresh programs in five families (scalar, vector, mixed, dependency-heavy and memory-aliasing) only after each version was frozen. The direct predecessor of our final compiler, which runs the same catalogue and the same depth-first search, was compared with the classical compiler on 200 such programs with a 0.1 s budget: its geometric-mean J was 14.4% lower (a descriptive comparison, without an interval), with 146 programs better, 39 equal and 15 worse. An earlier version with the same catalogue, on another fresh cohort of 200 programs, was 10.58% lower (98.33% interval 8.38% to 12.84%), with 138 better, 42 equal and 20 worse. The final compiler adds only the engineering refinement of Section 3, which makes the same decisions at equal work; it was not itself re-measured against classical on a fresh cohort, and we do not multiply results across rounds.

5.3 What it costs

Compile time is the price. The classical compiler compiles a public program in a median of 0.269 ms. Ours is designed to use its budget: its median compile call is 100.7 ms, and the slowest of its 40 public runs took 0.157 s as a whole process, including interpreter start. On the fresh cohort of round one, the search took 76.20 times the classical compile time. The grader allows 20 s per program, so both are far inside the limit; the difference is that the classical compiler leaves almost all of that allowance unused, while ours converts a small part of it into better output.

6 Why this approach

It optimises what is scored. Every question the search asks is framed in terms of $J = C \times S$, the quantity the score rewards. Trades between time and memory are therefore found, not forbidden.

It explains itself. Every option the compiler rejects carries a certificate that states its cause, and every accepted change is validated by the challenge’s own checker before it is kept. One can ask of any output why a cycle was not used or why a value moved, and receive a concrete answer.

It proves when to stop looking. When a window’s box is empty, or its exact search finishes without an improvement, that window provably has nothing better to offer given the rest of the program. Effort goes to windows that can still pay.

It is anytime and budget-aware. A correct answer exists after Step 1, and every later round only improves it, so the compiler can be stopped at any moment with a valid schedule. A larger budget buys more quality; a smaller one still returns something good. On the public programs even a budget of 0.01 s scores 2.135275.

It generalises by construction. Nothing refers to program names, operation identifiers or constants; the catalogue is derived from the structure of each program. For a company that compiles whole models into single kernels, where a scheduler must place work in time and memory together, a search that can reopen any group of decisions and explain each change seems to us the right foundation to build on.

7 Correctness and reproducibility

The submitted compiler is a single standard-library Python file that exposes `compile_program` and follows the challenge’s command-line contract. It passes the unchanged public test suite. Beyond the public programs it was run on a corpus of 142 programs and 277 cases, including regression, generated and stress programs that fill the 256-word scratchpad, and passed every case. All 1,248 command-line runs of the exported compiler on fresh and public programs were re-validated with the challenge’s own evaluator. Every number in this report is generated from frozen result files by the report’s build script, which re-derives each value from its source and refuses to compile if any digit in the text is typed by hand.

8 Limitations

The public programs were visible during development, and there is no result on the private grader. The comparison with classical on unseen programs was made for the direct predecessor of the final compiler, not for the final build itself. All timings come from one local machine. Under a real clock, a faster search can take a different path and occasionally end at a worse answer: at 0.1 s the final version was better than its predecessor in 123 paired runs, equal in 871 and worse in 6, so quality claims are about the distribution, not about every program. Finally, the classical compiler remains several hundred times faster to compile (median 0.269 ms against 100.7 ms); if compile time mattered more than output quality, it would be the better choice.

Tools and assistance

Claude Code assisted with implementation, review and the preparation of this report; every claim was checked by running the compilers and the challenge’s validator.

References

- [1] Luminal. *Compiler take-home*, pinned revision 573b8a85f4bdb8c3d8ba9f180d5f98dac875c902. <https://github.com/luminal-ai/interview>.
- [2] 3Blue1Brown Talent, *Luminal: AI inference compilers*. <https://www.3blue1brown.com/talent/>.
- [3] R. L. Graham, *Bounds on multiprocessing timing anomalies*, SIAM Journal on Applied Mathematics 17(2), 416–429 (1969).
- [4] P. B. Gibbons and S. S. Muchnick, *Efficient instruction scheduling for a pipelined architecture*, Proc. SIGPLAN ’86 Symposium on Compiler Construction, 11–16 (1986).
- [5] M. Poletto and V. Sarkar, *Linear scan register allocation*, ACM TOPLAS 21(5), 895–913 (1999).
- [6] P. Shaw, *Using constraint programming and local search methods to solve vehicle routing problems*, CP98, LNCS 1520, 417–431 (1998).
- [7] R. Castañeda Lozano, M. Carlsson, G. Hjort Blindell and C. Schulte, *Combinatorial register allocation and instruction scheduling*, ACM TOPLAS 41(3), Article 17 (2019).

- [8] O. Ohrimenko, P. J. Stuckey and M. Codish, *Propagation via lazy clause generation*, Constraints 14(3), 357–391 (2009).
- [9] H. Zenil, N. A. Kiani, A. A. Zea and J. Tegnér, *Causal deconvolution by algorithmic generative models*, Nature Machine Intelligence 1, 58–66 (2019).
- [10] H. Zenil, N. A. Kiani, F. Marabita et al., *An algorithmic information calculus for causal discovery and reprogramming systems*, iScience 19, 1160–1172 (2019).
- [11] A. Hernández-Espinosa, H. Zenil, N. A. Kiani and J. Tegnér, *Estimations of integrated information based on algorithmic complexity and dynamic querying*, arXiv:1904.10393 (2019).